Computer Architecture

Final Project

Computer Architecture 2026 (Prof. Chih-Wei Liu)

Overview

- Part 1 – Scalar Baseline
- Part 2 – RVV Vector Reduction
- Part 3 – SIMD-like RVV Parallelization
- Part 4 – CUDA SIMT Implementation
- Part 5 – Multi-pattern GPU Parallelism

Tools and Environment

- Tools
  - gem5
  - CUDA Toolkit
- Development Environment
  - Docker (Linux)
- Hardware Platform
  - Host CPU
  - NVIDIA GPU

Goal

- Investigate how modern architectures exploit parallelism.
- Explore scalar, vector, SIMD, and SIMT execution models.
- Program vector processors using RVV and GPUs using CUDA.
- Analyze and compare performance behavior across different architectures and implementations.

What is gem5 simulator ?

- A simulator for computer-system architecture research.
- Can help investigate impact of microarchitecture to applications.
- Cache size, frequencies, memory model, etc. could all be configured.
- Support scalar RISC-V and RISC-V Vector Extension execution.

What information can we get?

- Timing, memory bandwidth, and details on executed instructions.

```
simSeconds                                   0.207070                       # Number of seconds simulated (Second)
simTicks                                207070462000                       # Number of ticks simulated (Tick)
finalTick                               207070462000                       # Number of ticks from beginning of simulation (restored from checkpoints an
d never reset) (Tick)
simFreq                                 1000000000000                       # The number of ticks per simulated second ((Tick/Second))
hostSeconds                                    44.88                       # Real time elapsed on the host (Second)
hostTickRate                             4614216466                       # The number of ticks simulated per host second (ticks/s) ((Tick/Second))
hostMemory                                   390260                       # Number of bytes of host memory used (Byte)
simInsts                                    76762034                       # Number of instructions simulated (Count)
simOps                                      76762054                       # Number of ops (including micro ops) simulated (Count)
hostInstRate                                 1710497                       # Simulator instruction rate (inst/s) ((Count/Second))
hostOpRate                                   1710494                       # Simulator op (including micro ops) rate (op/s) ((Count/Second))
system.clk_domain.clock                         1000                       # Clock period in ticks (Tick)
system.cpu.numCycles                        414140924                       # Number of cpu cycles simulated (Cycle)
system.cpu.cpi                               5.394714                       # CPI: cycles per instruction (core level) ((Cycle/Count))
system.cpu.ipc                               0.185367                       # IPC: instructions per cycle (core level) ((Count/Cycle))
```

```
system.cpu.commitStats0.committedInstType::No_OpClass           50      0.00%      0.00% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::IntAlu     29757469     38.76%     38.76% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::IntMult           73      0.00%     38.76% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::IntDiv      4000040      5.21%     43.97% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::FloatAdd      2000000      2.61%     46.58% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::FloatCmp            2      0.00%     46.58% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::FloatCvt      4000022      5.21%     51.79% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::FloatMult           32      0.00%     51.79% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::FloatMultAcc      4000000      5.21%     57.00% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::FloatDiv            0      0.00%     57.00% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::FloatMisc           1      0.00%     57.00% # Class of committed instruction. (Count)
system.cpu.commitStats0.committedInstType::FloatSqrt           0      0.00%     57.00% # Class of committed instruction. (Count)
```

Setup for gem5

1. Select a directory you want to link and Launch a Container
   - **Only needed the first time**
   - docker run -it --name ca-fp-gem5 -v [your_path]:/workspace \
     -w /workspace weisheng505/gem5-rvv-image:v1
2. If you want to restart the container, run
   - docker start -ai ca-fp-gem5

Part 1 - Scalar Baseline

- Propose a computationally intensive and highly parallelizable algorithm.
  - Toy examples or overly simple algorithms are not allowed.
- Examples
  - Image processing
    - Image filters, Image Transform, Feature Extraction, …
  - DSP-related numerical computing
  - Machine learning / Deep learning

Part 1 - Scalar Baseline

- Requirement
  - At least one part of the algorithm must contain nested loops, where some computation within each iteration follows the form below.
  - $f(a_i[1], b_i[1]) + f(a_i[2], b_i[2]) + \ldots + f(a_i[n], b_i[n])$ for the i-th iteration
  - f must be implementable using basic arithmetic operations $(+, -, \cdot, /)$
    - For example, $a \cdot b$, $a \cdot b + c$, $(a - b)^2$, $a^2 + b^2$, ...

Part 1 - Scalar Baseline

- Implementation
  - C/C++
  - You may use g++ to verify your code before gem5 simulation.
- Compile the program with the RISC-V toolchain and execute in gem5.
  - Use the Makefile we provided. (compiled with riscv64-linux-gnu-g++)
  - make
- You can measure the simSeconds, simInsts, numCycles, CPI, …
  - Check the output file m5out/stats.txt
  - If you want to clean up the log files, run make clean.

RISC-V Vector Extension (RVV)

- Vector ISA extension for the RISC-V architecture.
- Introduces vector registers to accelerate data-parallel workload
  - `v0 ~ v31`
- Each vector register can contain multiple elements
  - `v1 = [a1, a2, a3, a4, ...], v2 = [b1, b2, b3, b4, ...], ...`
- A vector instruction can perform an arithmetic operation to all elements in parallel
  - E.g., vfmul.vv → `[a1*b1, a2*b2, a3*b3, a4*b4, ...]`

RVV Instruction Naming Convention

- RVV instructions usually follow <operation>.<format>
  - For example, vfmul.vv, vfadd.vf, vadd.vx, vredsum.vs
- The suffix describes the operand types
  - .vv = Vector op Vector
  - .vf = Vector op Scalar FP
  - .vx = Vector op Scalar Integer
  - .vi = Vector op Immediate
  - .v = Vector load/store
  - .vs = Vector to Scalar Reduction (e.g., summation)

Example RVV Instructions

RVV Instruction	Description	Meaning
vsetvli rd, rs1, vtypei	configure vector length	rd = new vl, rs1 = given vl, vtypei = type
vle32.v vd, (rs1)	vector load FP32 elements	load vl FP32 elements from memory into vd
vse32.v vs3, (rs1)	vector store FP32 elements	store vl FP32 elements from vs3 into memory
vfadd.vv vd, vs2, vs1	vector FP add	vd[i] = vs2[i] + vs1[i]
vfmul.vv vd, vs2, vs1	vector FP multiply	vd[i] = vs2[i] * vs1[i]
vfmacc.vv vd, vs1, vs2	vector FP mult-acc	vd[i] = vd[i] + vs1[i] * vs2[i]
vfmacc.vf vd, rs1, vs2	vector FP mult-acc with scalar	vd[i] = vd[i] + f[rs1] * vs2[i]
vmv.v.i vd, imm	move immediate to vector	vd[i] = imm
vfmv.v.f vd, rs1	move FP register to FP vector	vd[i] = f[rs1]
vfmv.f.s rd, vs2	move FP vector to FP register	f[rd] = vs2[0]
vfcvt.f.x.v vd, vs2	vector signed INT to FP convert	vd[i] = float(vs2[i])
vfredusum.vs vd, vs2, vs1	vector FP reduction unordered sum	vd[0] = vs1[0] + sum(vs2[i])
vredsum.vs vd, vs2, vs1	vector INT reduction sum	vd[0] = vs1[0] + sum(vs2[i])

Example RVV Instructions

- For more information, see *riscv-v-spec-1.0.pdf*.

```
# Vector unit-stride loads and stores

# vd destination, rs1 base address, vm is mask encoding (v0.t or <missing>)
vle8.v    vd, (rs1), vm  #    8-bit unit-stride load
vle16.v   vd, (rs1), vm  #   16-bit unit-stride load
vle32.v   vd, (rs1), vm  #   32-bit unit-stride load
vle64.v   vd, (rs1), vm  #   64-bit unit-stride load

# vs3 store data, rs1 base address, vm is mask encoding (v0.t or <missing>)
vse8.v    vs3, (rs1), vm  #    8-bit unit-stride store
vse16.v   vs3, (rs1), vm  #   16-bit unit-stride store
vse32.v   vs3, (rs1), vm  #   32-bit unit-stride store
vse64.v   vs3, (rs1), vm  #   64-bit unit-stride store
```

Samples for RVV instructions in C

- RVV instructions can be embedded into C/C++ code.

```
asm volatile(
    // VL(vector length) = t0 = 4,
    // SEW = 32 (32-bit per element), LMUL = 1 (1 vector register per op)
    "li t0, 4\n\t"
    "vsetvli zero, t0, e32, m1, ta, ma\n\t"
    "vle32.v v1, (%[A])\n\t"  // v1 = A[k][0,1,2,3]
    "vle32.v v2, (%[B])\n\t" // v2 = B[i][0,1,2,3]
    "vfmul.vv v3, v1, v2\n\t" // v3 = v1 * v2
    "vmv.v.i v0, 0\n\t"  // v0 = 0.0f vector for reduction initial value
    "vfredusum.vs v4, v3, v0\n\t" // v4 = sum(v3)
    "vfmv.f.s ft1, v4\n\t"  // ft1 = v4[0] (the result of reduction)
    "fsw ft1, (%[out])\n\t"
    :
    : [A] "r"(A),
      [B] "r"(B),
      [out] "r"(out)
    : "t0", "ft0", "ft1", "memory"
);
```

Parameters

A: const float*

B: const float*

out: float*

Result

*out = A[0]*B[0] + A[1]*B[1] + A[2]*B[2] + A[3]*B[3];

You may add outer loops to let the RVV assembly block run iteratively

Part 2 - RVV Vector Reduction

- Vectorize the computation of each iteration
  - $f(a_i[1], b_i[1]) + f(a_i[2], b_i[2]) + \ldots + f(a_i[n], b_i[n])$
  - If your n is too large, you may partition the summation into blocks
- You are asked to
  1. Use RVV vectors to map v1 = $a_i[1] \sim a_i[n]$ and v2 = $b_i[1] \sim b_i[n]$.
  2. Perform vectorized arithmetic operations for f.
  3. Use vector reduction instruction to sum up all values in the vector.
- Note
  - You are required to use Vector Reduction Operations.

Part 2 - RVV Vector Reduction

- Compile the program with the RISC-V toolchain and execute in gem5.
  - make
- Obtain simSeconds, simInsts, numCycles, CPI, overallMissRate
  - Check the output file m5out/stats.txt
- Does the reduction in cycles or instructions match your expectations?
- Compare the simulation results with the baseline.

Part 3 - SIMD-like RVV Parallelization

- Apply k-way loop unrolling.
  - For every group of k iterations, we compute

    $f(a_1[1], b_1[1]) + f(a_1[2], b_1[2]) + \ldots + f(a_1[n], b_1[n])$

    $f(a_2[1], b_2[1]) + f(a_2[2], b_2[2]) + \ldots + f(a_2[n], b_2[n])$

    $\ldots$

    $f(a_k[1], b_k[1]) + f(a_k[2], b_k[2]) + \ldots + f(a_k[n], b_k[n])$
  - Choose the block size k by yourself
  - If your n is too large, you may partition the summation into blocks
  - Perform across-k SIMD parallelization. Each vector lane corresponds to one iteration within a group.

Part 3 - SIMD-like RVV Parallelization

- You are asked to
  1. Use RVV vectors to map v1 = $a_1[j] \sim a_{\boldsymbol{k}}[j]$ and v2 = $b_1[j] \sim b_{\boldsymbol{k}}[j]$
  2. Perform vectorized arithmetic operations for function f and accumulation in a SIMD-like manner.
- Note
  - **Do NOT** use Vector Reduction Operations
    - Vector reduction operations are unnecessary here and only make things harder. If you use it, it means you have misunderstood the objective of this part.
  - Strided memory access will be required
    - E.g., vlse32.v, vsse32.v

Part 3 - SIMD-like RVV Parallelization

- Compile the program with the RISC-V toolchain and execute in gem5.
  - make
- Obtain simSeconds, simInsts, numCycles, CPI, overallMissRate
  - Check the output file m5out/stats.txt
- Does the reduction in cycles or instructions match your expectations?

RVV Instruction	Description	Meaning
`vle32.v` `vd, (rs1)`	vector load FP32 elements	load `vl` FP32 elements with unit stride
`vse32.v` `vs3, (rs1)`	vector store FP32 elements	store `vl` FP32 elements with unit stride
`vlse32.v` `vd, (rs1), rs2`	strided vector load FP32 elements	load `vl` FP32 elements with byte stride `rs2`
`vsse32.v` `vs3, (rs1), rs2`	strided vector store FP32 elements	store `vl` FP32 elements with byte stride `rs2`

Part 3 - SIMD-like RVV Parallelization

- Compare the simulation details against the previous implementations from part 1 and 2.
- Think:
  - 為什麼 cycle reduction 與 instruction reduction 不完全相同？
  - 當 k 愈大，performance 愈好嗎？為什麼？
  - 為甚麼 Part 3 不需要 reduction operations？與 Part 2 相比有什麼差異或取捨？

What is CUDA ?

- A C-like programming model for GPU acceleration.
- NVIDIA's parallel computing platform for heterogeneous computing.
- Can execute massively parallel workloads with arrays of GPU threads.
- The CUDA compiler (nvcc) generates both CPU host code and GPU executable code.

What information can we get?

- From PTXAS
  - Registers per thread
  - Shared memory per block
  - spill stores, spill loads
- From PTX
  - Load/store instruction
  - Arithmetic instruction
  - Shared memory usage
  - Global memory usage

What information can we get?

- Profile runtime GPU behavior using Nsight Compute (ncu).
  - Memory throughput
  - Cache throughput
  - SM utilization
  - Compute (SM) throughput
  - Occupancy
  - Active warps per SM
  - Waves per SM

Setup for CUDA Toolkit

1. Check whether you have an NVIDIA GPU
   - nvidia-smi
2. Select a directory you want to link and Launch a Container
   - **Only needed the first time**
   - docker run -it --gpus all --name ca-fp-cuda -v [your_path]:/workspace \
     -w /workspace weisheng505/cuda-env:v1
   - If you do not have an NVIDIA GPU, remove the --gpus all option.
3. If you want to restart the container, run
   - docker start -ai ca-fp-cuda

If you don't have an NVIDIA GPU, you may execute on Google Colab.

Setup for CUDA Toolkit

4. After entering the container, verify that the GPU is correctly available.
   - nvidia-smi
5. Choose a matching target architecture for nvcc
   - For example,
     - RTX 20-series / T4 → sm_75
     - RTX 30-series → sm_86
     - RTX 40-series → sm_89
   - For more details:
     https://developer.nvidia.com/cuda-gpus

If you don't have an NVIDIA GPU, you may execute on Google Colab.

CUDA programming

- CUDA programs consist of
  - **Host code**: runs on CPU
  - **Device code (kernel)**: runs on GPU
- CUDA Execution Hierarchy
  - **Thread**: The smallest execution unit in CUDA.
  - **Warp**: A group of 32 threads executed together in SIMT fashion.
  - **Block**: A group of threads that can synchronize and share shared memory.
  - **Grid**: A collection of blocks launched by a kernel.

CUDA programming

- Basic CUDA Components
  - __global__
    - Declares a GPU kernel function callable from CPU host code
  - cudaMalloc((void**)&d_ptr, size_in_bytes)
    - Allocates memory on GPU device memory.
  - cudaMemcpy(destination, source, size, direction)
    - Copies data between CPU and GPU memory
- For more details for CUDA programming, refer to *Tutor1_basic_CUDA_programming.pptx*

Part 4 - CUDA SIMT Implementation

- Implement a CUDA version using the SIMT execution model on a GPU.
  - Each thread computes one output task.
  - CUDA groups 32 threads into a warp and executes them together.
- Assign the iteration index
  - threadIdx.x gives the local thread index inside a block.
  - For multiple blocks, you may use threadIdx.x, blockIdx.x and blockDim.x
- Load any reused values into shared memory if necessary.
  - Use __shared__

Part 4 - CUDA SIMT Implementation

Note: We recommend you specify your GPU architecture with -arch=sm_xx option when using nvcc.
e.g., -arch=sm_75, -arch=sm_86, -arch=sm_89, etc.

- Compile with nvcc and Execute
  - nvcc -Xptxas -v [your_program].cu -o main -O2 -arch=sm_[xx]
  - ./main
- You may check your PTXAS information after compilation.

Part 4 - CUDA SIMT Implementation

- Generate PTX
  - nvcc --ptx [your_program].cu -O2 -arch=sm_[xx]
- Analyze PTX Instructions
  - Check the output file [your_program].ptx
  - Is your program more memory-intensive or compute-intensive?
- Observe GPU execution behavior
  - ncu --set basic ./main

Part 4 - CUDA SIMT Implementation

- You can change the number of Threads Per Block and analyze its impact on performance and occupancy.
  - You may measure GPU kernel runtime using either APIs in <cuda_runtime.h> (recommended) or Nsight Compute.

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
cudaEventRecord(start);
...<<< , >>>(...);
cudaEventRecord(stop);
cudaEventSynchronize(stop);
float ms;
cudaEventElapsedTime(&ms, start, stop);
printf("Kernel time = %f ms\n", ms);
```

If you want to measure the runtime on CPU for the baseline, use <chrono>

```
auto st = std::chrono::high_resolution_clock::now();
...(...);
auto ed = std::chrono::high_resolution_clock::now();
std::chrono::duration<double> elapsed = ed - st;
printf("CPU time: %f ms\n", elapsed.count() * 1000);
```

Part 4 - CUDA SIMT Implementation

- You are encouraged to experiment with different thread memory access pattern (不同的資料切割方法).
- Think:
  - Shared memory 在什麼情況下比較有效？使用與否會造成甚麼影響？
  - Block size (Threads Per Block) 愈大，performance 一定愈好嗎？
  - Block size 會影響 GPU 同時執行的 threads/warps 數量嗎？ occupancy 如何改變？
  - Occupancy 的改變會如何影響 GPU latency hiding 能力？

Part 5 - Multi-pattern GPU Parallelism

- The testbenches for the previous parts process only one input pattern.
- In this part, we use the GPU to process multiple input patterns.
  - Exploits parallelism across independent patterns.
  - You may write another multi-pattern baseline code for verification.
- You may use a 2D CUDA Grid Mapping
  - x-dimension: output / thread index
  - y-dimension: pattern index
  - Declare the 2D grid using dim3 grid(..., ...)
  - Use blockIdx.y to obtain the pattern index.

Part 5 - Multi-pattern GPU Parallelism

- Compile with nvcc and Execute.
  - nvcc -Xptxas -v [your_program].cu -o main -O2 -arch=sm_[xx]
  - ./main
- You may check your PTXAS information after compilation.
- Generate PTX
  - nvcc --ptx [your_program].cu -O2 -arch=sm_[xx]
- Analyze PTX Instructions
  - Check the output file [your_program].ptx
  - Is your program more memory-intensive or compute-intensive?

Part 5 - Multi-pattern GPU Parallelism

- Observe GPU execution behavior
  - ncu --set basic ./main
- Think:
  - 為什麼 Part 5 會比 Part 4 更適合 GPU？
  - 為什麼 GPU 必須依賴大量 threads/warps 才能發揮 scalability 的效益？
  - 增加 pattern 數量後，performance 是否持續提升？會隨數量線性成長嗎？
  - 增加 pattern 數量如何影響 occupancy 與 latency hiding 能力？進而對 GPU utilization 造成什麼影響？

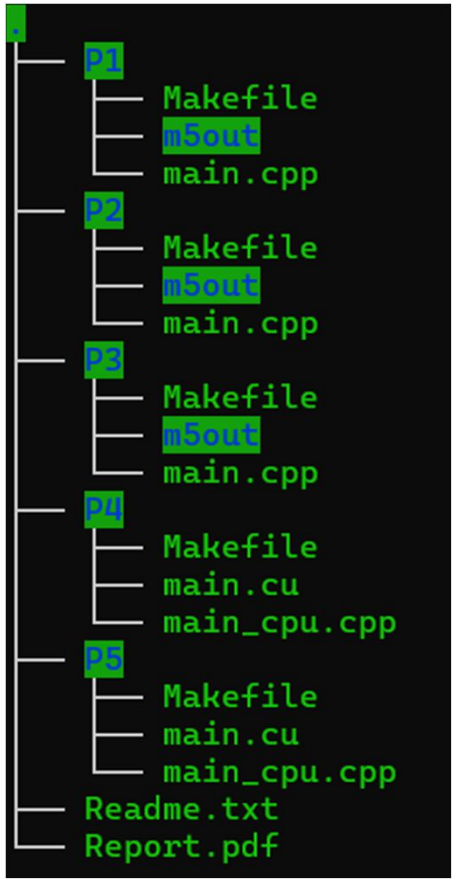
分組表單

- 最多三人一組。
- 找好組員請至
  https://docs.google.com/spreadsheets/d/1O2isRS5uJJcNO23I7dSzE3UhE7-O2iSSOVwtky0TwkQ/edit?usp=drive_link
  填寫組員與自己學號。

繳交方式

- 繳交檔案
  - 學號1_學號2 _學號3.zip
    (e.g., 0650001_0650002_0650003.zip)
  - 壓縮檔至少包含
    - 各 Part 的主程式 (檔名沒有限制)
    - Report.pdf
    - 其他執行程式所需要的額外檔案
      (如：標頭檔、額外套件、輸入資料、預處理等)
    - 於 Readme.txt 說明如何執行同學的程式。
- 繳交平台：e3p 教學平台

example



Report

僅供參考，同學可以自行增減

- 環境
- 檔案說明
- 演算法內容
- 實現方法
- 模擬結果說明
- 討論與比較
- 結論

Reminder

1. GPU 請註明使用型號。
2. gem5 simulated time 與實際 CPU/GPU runtime 是兩種完全不同的概念，不要搞混。
3. 請確保演算法的運算結果會被輸出或參與後續簡單驗證/計算，避免 compiler 將未使用的運算移除掉。
4. 不要只有數據，請說明 performance behavior 並分析其原因。